

awesome-llm-apps 小白指南

140,000+ Star 的 AI 智能体百宝箱 —— 88 个能直接跑的现成智能体,从"会说"到"会做"

★ 140.4k Stars • 20.6k Forks • Apache-2.0 免费商用 • 每周更新

仓库:github.com/Shubhamsaboo/awesome-llm-apps · 本指南实验环境:Windows 11 + Python 3.14 + agno 3.0.11 + GLM-4.6(API 兼容接入点)

学习日期:2026-10-01 · 属于「400+ 子项目学习计划」· 知识已入 Obsidian vault

①这是什么

②Agent≠聊天机器人

③工具调用循环

④仓库地图

⑤RAG

⑥多智能体团队

⑦MCP

⑧Agent Skills

⑨更多花样

⑩agno框架

⑪记忆

⑫没OpenAI key怎么跑

⑬能力与用法

🔗 动手实验

Q1 这是什么?一句话能说清吗?

awesome-llm-apps 是一本「AI 智能体菜谱大全」:100+ 个手工构建、端到端测试过的智能体示例,克隆下来 30 秒就能跑,免费、开源、可商用。

awesome-llm-apps(直译"很棒的 LLM 应用")不是一款软件,而是一个**示例合集仓库**——类似"预制菜大礼包":每个文件夹就是一道完整的菜(一个能跑的智能体应用),带代码、带说明书(README)、带食材清单(requirements.txt)。

📖 类比你学过的「**程序化建模**」:菜谱而非泥人。这个仓库也一样的思想——**不教你从零捏智能体,而是给你 88 份验证过的"菜谱"**,你挑一份改改就能上桌。vault 里「模板复用降低试错成本」这条经验在这里完全适用。

作者 Shubhamsaboo(团队 Unwind AI),口号是 "**Clone it, ship it, sell it**"(克隆、上线、拿去卖都行,Apache-2.0 协议),支持 Claude / GPT / Gemini / DeepSeek / Llama / Qwen 等几乎所有主流模型。2026 年拿下 Trendshift 当日第一。



图1:一个仓库 = 一整箱现成的智能体应用

Q2 AI 智能体(Agent)到底是什么?和聊天机器人差在哪?

聊天机器人只会「说」,智能体会「说 + 做」:它能调用工具、查资料、执行动作,自己决定下一步干什么。

AI 智能体(Agent):以大语言模型为“大脑”,配上工具当“手脚”,再给一个自主循环让它“想了就做、做完再想”的程序。

比喻:聊天机器人像**电话客服**——你问它答,答完就挂,办不了任何实事。智能体像**有手有脚的助理**——你说“帮我订下周三的机票”,它自己查航班、比价格、打开订票页面、下单,然后回来汇报。

落回术语:区别的本质是——聊天机器人的输出只有文字;智能体的输出可以是「**工具调用请求**」,由程序真正执行,再把结果喂回大脑继续决策。

智能体 = 大脑(LLM) + 手脚(工具 Tools) + 循环(自主决定做几步)

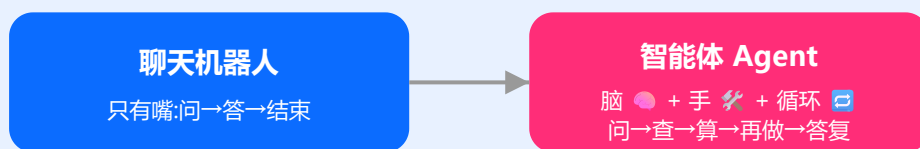


图2:从「会说」到「会做」

Q3 它是怎么「自己动手」的?—— 工具调用循环(全仓库的核心原理)

关键机制叫「工具调用 / Function Calling」:LLM 不执行代码,它只「点菜」(输出函数名+参数的JSON),真正下厨的是框架;菜端回来,它再决定下一口吃什么。

工具调用(Function Calling):让 LLM 从一份"工具菜单"里选择"我要调哪个函数、传什么参数",由外部程序代为执行并把结果还给 LLM 的机制。



图3:工具调用循环 —— 本仓库所有 88 个示例的共同心脏

✅ **已实测验证**(agno 3.0.11 + GLM-4.6):问"光一年走多少公里",日志清楚显示 calculator(expression=3e5 * 365 * 24 * 3600),然后 LLM 用返回值组织出「约 9.4608×10¹² 公里」;问"现在几点",它调用 now_time() 拿到真实时间。两问都没心算、没编时间——**菜单机制生效了**。

🔗 **连接你的知识库**:这个「生成→执行→检查→修正」的循环,和 [[质检Gate与自我纠错循环]] 是同族思想;每一步工具结果就是一次客观的 Gate 检查,LLM 据此决定继续还是收工。区别在于:Gate 循环的裁判是确定性脚本,这里的裁判是 LLM 自己读结果后决策。

Q4 仓库里都有什么?(88 个示例的地形图)

按「学什么」分四大区:单智能体(入门)、多智能体(进阶)、能力插件(RAG/MCP/Skills/记忆)、专题应用(语音/UI/游戏/微调)。

分区(实测目录)	数量	代表示例	学到什么
 starter_ai_agents	17	旅行助手、数据分析、CSV 问答	单文件智能体,一个 API key 就能跑
 advanced_ai_agents	15+	深度调研、欺诈调查、VC 尽调团队	生产级:记忆、多步推理、团队协作
 agent_skills	9	项目墓地、范围蔓延检测器	给编程智能体装"技能包"
 rag_tutorials	24	纠错式 RAG、本地 RAG、知识图谱 RAG	RAG 从最简链到智能体化
 mcp_ai_agents	6	浏览器、GitHub、Notion 智能体	用 MCP 协议连接外部世界
 always_on_agents	2	HN 日报、依赖发布雷达	定时/事件驱动的后台智能体
 voice /  生成式UI /  游戏对战	10+	语音理赔、聊天生成看板、双 LLM 下棋	智能体的语音与图形界面形态
 记忆 /  Chat with X /  微调 /  框架速成	15+	带记忆的 ArXiv 助手、Chat with PDF、LoRA 微调	长对话、知识问答、模型定制

 **怎么逛:**把 starter 区当「必修课」(17 个全部单文件),rag_tutorials 当「专业课」,其余按需求「选修」。每周还会上新。

Q5 RAG 是什么?为什么「让 AI 读我的 PDF」不能直接把 PDF 塞给它?

RAG(检索增强生成)= 先从你的资料里「检索」出最相关的几段,再把这几段塞进提示词让 LLM 「生成」答案。解决的是:资料太多塞不下 + 模型没见过你的私有资料。

直接塞整本 PDF 有三个问题:① **上下文窗口有限**(模型一次能读的字数有上限);② **贵**(按字数收费);③ **大海捞针**(塞得越多,模型越容易漏掉关键句)。

嵌入(Embedding):把一段文字变成一串数字(向量),意思越相近的段落,向量越接近——于是"找相关段落"变成"找最近的向量"。

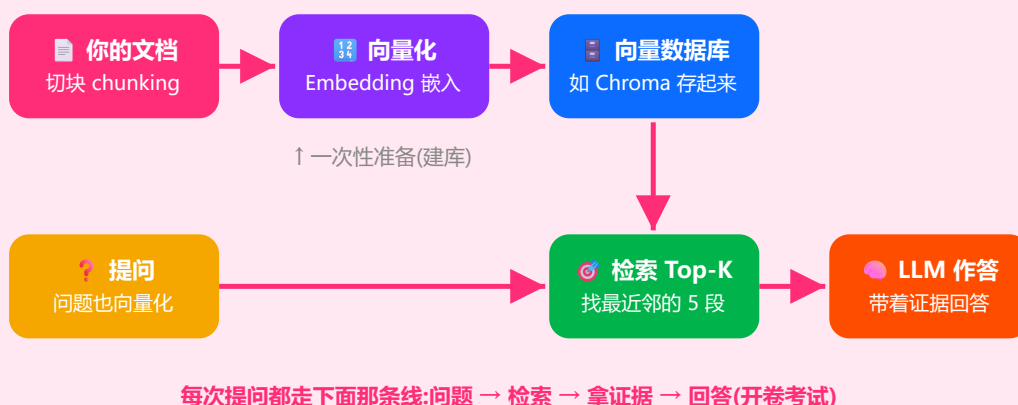


图4:RAG 两阶段 —— 建库一次,检索无数次

比喻:让 LLM 直接读整本书,是逼它闭卷硬背;RAG 是**考试前帮它做好一盒索引卡片**,考到哪个知识点就抽出对应的卡片放它面前——它只需要读那几张卡片就能作答。**落回术语:**卡片盒=向量数据库,做卡片=Embedding 建库,抽卡片=Top-K 检索。

✅ **已实测源码**(repo/rag_tutorials/rag_chain/app.py,199 行):Chroma 向量库 → `db.as_retriever(k=5)` → 提示词模板 → LLM → 输出,和图 4 一一对应。该区 24 个教程 = 这条链的无数种升级(检索质量自评、查不到就转网搜、多模态、带引用.....)。

Q6 多智能体团队(Multi-Agent Team)为什么要「分工」？

一个智能体什么都管,指令会互相打架、工具越挂越多;拆成各管一摊的专员,再让一个「队长」汇总,效果和可维护性都更好。

 **比喻:**去医院不会挂一个「全科包治百病」号,而是**分诊台(Team)**→ **各科专家(member agents)**。每位专家只带自己科室的工具(化验单、CT 机),诊断更准;分诊台负责听你描述、决定叫哪些科、最后汇总病历。**落回术语:**member=有专属工具和指令的子智能体,Team=带汇总模型的编排器。

✅ **已实测源码**(ai_finance_agent_team,约 60 行):web_agent(挂搜索引擎工具)+ finance_agent(挂股价工具)→ Team(members=[web_agent, finance_agent])。读股票分析问题,队长自动拆分"查新闻"和"查股价"两路,再合并成报告。



图5:60 行代码的金融分析团队(仓库真实示例)

Q7 MCP 是什么?为什么全网都在说?

MCP(Model Context Protocol,模型上下文协议)= AI 工具的「USB-C 接口标准」:工具做一次 MCP Server,所有支持 MCP 的智能体都能即插即用。

没有统一标准时:你有 M 个智能体、N 个工具(浏览器、GitHub、Notion.....),最坏要写 $M \times N$ 套对接代码。有了 **MCP**,工具方实现一次 MCP Server,智能体方实现一次 MCP Client,变成 $M+N$ 。

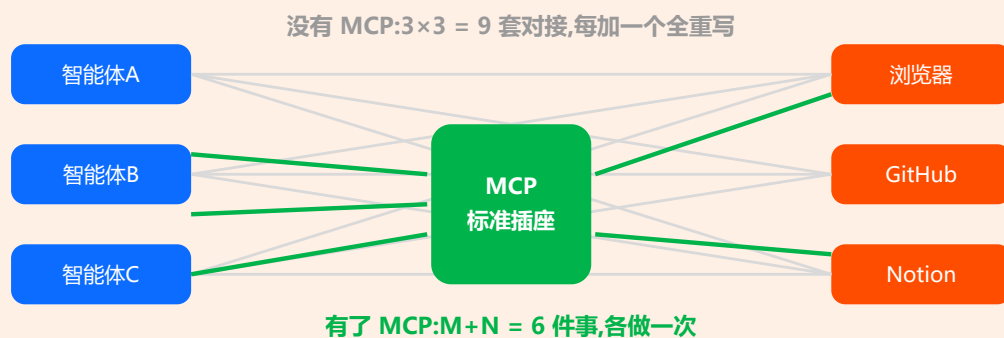


图6:MCP 把 $M \times N$ 变成 $M+N$

仓库的 `mcp_ai_agents/` 给了 6 个现成例子:自然语言开真浏览器、用嘴逛 GitHub 仓库、指挥 Notion。你现在用的 Claude Code 本身就是 MCP Client——这就是为什么"给 Claude 装个浏览器工具"只需要一行配置。

Q8 Agent Skills(技能包)和工具 / MCP 有什么区别?

工具是「函数代码」(给程序执行);Skill 是「方法论文档+脚本」(给 LLM 按需阅读照做);MCP 是「连接标准」。三者一层套一层,不冲突。

仓库 agent_skills/ 的 9 个技能用一条命令装进 Claude Code / Codex / Cursor:

```
npx skills add https://github.com/Shubhamsaboo/awesome-llm-apps/tree/main/agent_skills/project-graveyard
```

✅ 已实测目录结构(project-graveyard):SKILL.md(frontmatter 写 name + description,正文写"什么时候用、怎么做")+ scripts/(可执行脚本)+ references/(参考资料)。每个技能还要过安全+评测 CI 门禁。

🤖 你已经在用同款东西了:你每天用的 /learn-project skill 就是这个格式——一个 SKILL.md 加几个文件。换句话说: awesome-llm-apps 的 agent_skills 区,就是"别人写好的、可以直接 npx 装进你 Claude Code 的新技能"。

	工具 Tool	技能 Skill	MCP
本质	Python 函数(代码)	Markdown 说明书 + 脚本	连接协议标准
给谁用	框架在运行时执行	LLM 读入上下文后照做	Client 连 Server
类比	厨房里的锅	菜谱卡片	燃气接口标准

Q9 还有什么花样?(常驻、生成式 UI、语音、游戏、微调)

形态	是什么	仓库例子
 常驻智能体 Always-on	不是"你问它答",而是定时/事件驱动,主动干活推送结果	每天早上把 Hacker News 排成日报发 Slack; 盯依赖库发新版就提醒
 生成式 UI	智能体的回答不是一段字,而是 渲染的交互组件 :卡片、表单、图表	聊着聊着生成看板、仪表盘、行程编辑器
 语音智能体	实时语音进、语音出,能打断能接电话	保险理赔语音坐席(边听边看摄像头边记录)
 游戏对战	两个不同 LLM 各执一色对下,验证走子合法性	双模型国际象棋、井字棋
 微调	拿自己的数据继续训练开源小模型	Gemma 3 / Llama 3.2 的 4-bit LoRA 微调教程

其中「微调-LoRA」你已经在 changeheadanddance 项目(视频换人)接触过——同一个思想:**不换大脑,只插一张小小的"技能卡"**。

Q10 agno 框架是什么?为什么所有示例都用它?

智能体框架 = 脚手架:替你管「模型对接、工具注册、消息循环、会话存储」,你只写"用哪个脑、带哪些手、守什么规矩"三件事。

没有框架时,你要自己写:HTTP 调模型的代码、把工具清单翻译成 API 要的 JSON 格式、解析模型返回的"点菜单"、执行函数、把结果按格式塞回去、循环、存聊天记录.....约 200 行样板代码。用 agno 只要 6 行:

```
from agno.agent import Agent

agent = Agent(
    name="小助手",
    model=Claude(id="glm-4.6", ...), # ① 用哪个脑
    tools=[now_time, calculator],    # ② 带哪些手
    instructions=["计算必须用工具"], # ③ 守什么规矩
)
```

agno 还自带 Team(多智能体编排)、SqliteDb(会话/记忆存储)、AgentOS(一键起服务)。同类框架还有 LangChain/LangGraph、CrewAI、AG2;仓库的"框架速成课"教 Google ADK 和 OpenAI Agents SDK——概念互通,换框架只是换语法。

你写的:脑 + 手 + 规矩(6 行)

框架 agno:工具注册 · 消息循环 · 格式翻译 · 会话存储

模型 API(GLM / GPT / Claude / Llama ...任选)

图7:框架夹在「你的三行配置」和「模型 API」之间

Q11 「记忆(Memory)」是怎么回事?保存聊天记录不就是记忆吗?

聊天记录=本次对话的"工作记忆";记忆=跨会话提炼并记住的"用户画像与偏好"。前者每开新窗口就清零,后者写进数据库,下次见面还认识你。

🔗 **比喻:**只存聊天记录的助手像**金鱼**——这顿饭聊得再好,下次见面就不认识你了。有记忆的助手像**带笔记本的老管家**:服务完把你"不吃香菜、喜欢靠窗位"记进小本本(数据库),下次直接安排好。**落回术语:**短期记忆=上下文里的对话历史;长期记忆=框架从历史中提炼、存入 SqliteDb 等存储的结构化信息。

仓库写法就两行:db = SqliteDb(db_file="agents.db") + add_history_to_context=True——历史自动进库、自动带回上下文。memory 专区 6 个示例演示了"记住你的研究兴趣/旅行偏好/每人独立记忆"等玩法。

Q12 没有 OpenAI key,能用国产 / 本地模型跑这些示例吗?

能。三条路(本指南实测第一条):① 任何 Anthropic 兼容端点(如 GLM)→ agno 的 Claude 类 + 换 base_url;② OpenAI 兼容端点(DeepSeek/Qwen/硅基流动)→ agno 的 OpenAIChat 换 base_url;③ 完全本地 → Ollama + llama3.2/qwen,零 API key。

```
# 本指南实测通过:用 GLM 的 Anthropic 兼容端点跑 agno 智能体
from agno.models.anthropic import Claude

model = Claude(
    id="glm-4.6",
    api_key=os.environ["ANTHROPIC_AUTH_TOKEN"],
    client_params={"base_url": os.environ["ANTHROPIC_BASE_URL"]},
)
```

⚠ 实测踩坑记录(宝贵的三个)

- ① **agno 3.x 接口变了**:网上旧教程写 Claude(base_url=...) 会报 TypeError,3.x 要用 client_params={"base_url": ...};而仓库示例还是按 2.x 写的——**跑仓库示例前先 pip 装兼容版本或按新 API 小改。**
- ② **限流 429**:GLM 接入点约每分钟 1 次请求,连续两次调用要 time.sleep(65),否则报 rate_limit_error 1302。
- ③ **中文 Windows 的 GBK**:跑 Python 前先 setx PYTHONUTF8 1,否则中文输出乱码/假报错。

本地路线示例(仓库有大量 local_* 教程,零 key):Ollama(id="llama3.2")——先装 Ollama,再 ollama pull llama3.2(约 2GB),之后模型跑在你自己电脑上,不花钱、数据不出门。

Q13 这个项目给我什么?我拿它干什么、怎么挑怎么改?

一句话:88 个验证过的起点,让你做智能体应用时「改模板」而不是「从零写」。



跳过从零搭建 —— 每个示例都是能跑的完整应用,改提示词、换工具、换模型就是你的第一版。



四大件的最小实现教材 —— Agent 工具循环 / RAG / 多智能体 / MCP,每个概念都有从 30 行到生产级的渐进示例。



搭自己的自动化助手 —— 用国产/本地模型(已实测)做:CSV 数据问答、文档知识库、监控日报、桌游裁判.....



给编程智能体装技能 —— npx skills add 一条命令,把 9 个技能装进 Claude Code(和你自己的 skill 同格式,照着写就行)。



改造套路四步:①挑最接近需求的 starter → ②换 model 成自己有的 key → ③换/加 tools 成自己领域的函数 → ④改 instructions 立规矩。学会了,88 个示例 = 88 块乐高。



动手实验:从零跑起你的第一个 AI 智能体(全部命令已实测)

步骤 ① 建实验环境(虚拟环境 + 装 4 个包)

怎么操作

```
python -m venv .venv
```

```
.venv\Scripts\pip install agno anthropic streamlit icalendar
```

目的

虚拟环境隔离依赖,不污染系统 Python

实现了什么

装好 agno(智能体框架)、anthropic(官方客户端,agno 调模型要经过它)、streamlit(示例界面)、icalendar(示例用到)

怎么验证

```
运行 .venv\Scripts\python -c "import agno, anthropic; print(agno.__version__ if hasattr(agno, '__version__') else 'ok', anthropic.__version__)"
```

✅ 预期结果:ok 1.11.0(或 agno 3.0.11 / anthropic 1.11.0 这样的版本号),无报错

步骤 ② 配置模型接入(环境变量)

怎么操作 确认系统已有 `ANTHROPIC_BASE_URL` 和 `ANTHROPIC_AUTH_TOKEN` 两个环境变量(GLM 兼容端点);再执行 `setx PYTHONUTF8 1`

目的 让脚本知道往哪调模型、用什么钥匙;同时解决中文 Windows 的 GBK 乱码

实现了什么 任何 Anthropic 兼容端点都能被 agno 当作"Claude"使用——key 不写进代码,安全

怎么验证 `echo $env:ANTHROPIC_BASE_URL` 打印出你的接入点地址(token 不要打印出来)

✅ 预期结果:打印出 `https://.../api/anthropic` 之类的地址;没有这两个变量的话,去你的模型服务商后台申请

步骤 ③ 冒烟测试:先让"大脑"说一句话

怎么操作 写 `exercise\smoke_test.py`(指南附带,见右侧要点):定义 Agent(只给 model,不给工具),`agent.run("什么是 AI 智能体?")`,运行 `.venv\Scripts\python exercise\smoke_test.py`

目的 验证"模型接入"这一环,排除工具因素的干扰

实现了什么 第一次真正调通 LLM API;metrics 里能看到 input/output tokens 消耗

怎么验证 看终端输出

✅ 预期结果:打印一句中文定义,如「AI 智能体是一种能够自主感知环境、进行推理决策并采取行动以完成特定目标的智能系统」,以及 `RunMetrics(input_tokens=27, output_tokens=106, ...)`

步骤 ④ 主实验:给大脑装上「计算器 + 时钟」两只手

怎么操作 写 `exercise\my_first_agent.py`:定义两个普通 Python 函数 `now_time()` 和 `calculator(expression)`,放进 `Agent(tools=[now_time, calculator])`,问它两个问题,加 `show_tool_calls=True`;两个问题之间 `sleep 65` 秒(见坑②)。运行它。

目的 亲眼看到 Q3 的"工具调用循环"真实发生,而不是听人转述

实现了什么 LLM 自己决定调哪个函数、传什么参数;框架执行后它再用结果作答

怎么验证 输出里应出现 agno 画的 **Thinking / Tool Calls / Response** 三个框

✅ 预期结果:问题 1 出现 • `calculator(expression=3e5 * 365 * 24 * 3600)`,答复「约 9.4608×10^{12} 公里」;65 秒后问题 2 出现 • `now_time()`,答复真实当前时间(实测它还认出当天是国庆节!)

步骤 ⑤ 改造:发明一个你自己的工具(掷骰子)

- 怎么操作

写 exercise\my_tool.py:自己定义 roll_dice(sides, times)(用 random 真掷),挂进 Agent,instructions 里加一条"随机数必须调工具,禁止编造";问它"掷 2 个六面骰 + 1 个二十面骰"。
- 目的

体会 Q8 那句话:「工具就是普通 Python 函数」——你能定义任何函数,智能体就能用任何能力
- 实现了什么

你的第一个"自研工具";LLM 无法自己产生真随机数,必须依赖工具——这展示了工具的不可替代性
- 怎么验证

输出应出现 • roll_dice(sides=6, times=2) 等调用,且点数确实是随机的(多跑几次答案不同)

✓

预期结果:工具调用框列出掷骰调用,回复报出每次点数和总和;在 instructions 里禁止编随机数后,它不再自己编

⚠ 实验总坑位(再次提醒):① agno 3.x 用 client_params 传 base_url;② 该 API 限流约 1 次/分钟,连续调用 sleep 65;③ PYTHONUTF8=1。全部命令均于 2026-10-01 在 Windows 11 + Python 3.14 实测通过。



概念速查(已存入 Obsidian vault,可直接双链跳转)

概念	一句话	vault 笔记
AI 智能体 Agent	LLM 大脑 + 工具手脚 + 自主循环	概念/AI智能体Agent
工具调用 Function Calling	LLM 点菜(函数名+参数),框架下厨	概念/LLM工具调用
RAG 检索增强生成	先检索相关段落,再让 LLM 带证据作答	概念/RAG检索增强生成
多智能体团队	分工的子智能体 + 汇总的队长	概念/多智能体协作
MCP 协议	AI 工具的 USB-C 标准,M×N 变 M+N	概念/MCP模型上下文协议
Agent Skills	给 LLM 读的方法论包(SKILL.md 格式)	概念/AgentSkills技能包
智能体框架	管循环/注册/存储的脚手架,如 agno	概念/智能体框架